CPE 314 — Software Design

University of Cebu

Computer Engineering Department

Plate N Platter

Plan smarter. Eat better. Cook easier

Project Design Document

Group Name / No.:	PNP/1
Members:	Luistro, Kimberly - Team Leader
	Librea, Nesley Cate - UI
	Nadela, Venice - QA
	Tapan, Yvonne Mae - UI
	Quisagan, Kirk Ryan - Developer
	Tamidles, Ken - DB Admin
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	V 0.4 / November 11, 2026

Table of Contents

Revision History

1. Introduction & Project Overview 1

2. Preliminary ER Sketch & Team Roles 2

2.1 Key Attributes and Relationships 2
2.2 Team Roles 4

3. System Architecture & Object-Oriented Design 5

3.1 Class Hierarchy 5
3.2 Abstract Class SavedRecipe and the PlanStore Interface 5
3.3 One Table, One Class 7
3.4 Single Responsibility of Each Class 8
3.5 Simpler, Smaller Code 10
3.6 User Interface Design 12
3.7 Forgot Password Flow 13
3.8 Design Rationale 14

4. Data Model & Database Design 15

4.1 Database Overview 15
4.2 Tables and Relationships 15
4.3 Core Table Schema 16
4.4 Query Design 17
4.5 Agreement with the ER Sketch 18

5. API Integration Design 19

5.1 The TheMealDB Calls 19
5.2 JSON Response 19
5.3 Parsing 20
5.4 Use in the App 20
5.5 Prep Time and Calories 21
5.6 Today's Recommendations 22
5.7 Error Handling and the Offline Message 23

6. Concurrency Design 25

6.1 Long-Running Operations 25
6.2 Threading Approach 26
6.3 How the UI Stays Responsive 26
6.4 Thread Safety 27

7. Data Mining & Visualization Design 28

7.1 Aggregation Step 28
7.2 Worked Example 29
7.3 Chart and Why It Was Chosen 29

8. Testing Summary 30

9. Individual Contributions 30

Appendix 31

A. References and API Documentation 31
B. Licensing Notes 31
C. Known Issues and Limitations 31

Revision History

Changes to this document, by version and date.

Table 1: Revision history

Version	Date	Author	Changes
0.1	October 22, 2026	GROUP PNP	Initial draft: Section 1 (Introduction & Project Overview) and Section 2 (Preliminary ER Sketch & Team Roles)
0.2	October 30, 2026	GROUP PNP	Section 3 (System Architecture & OOD) and Section 4 (Data Model & Database Design) added
0.3	November 6, 2026	GROUP PNP	Section 5 (API Integration), Section 6 (Concurrency) and Section 7 (Data Mining & Visualization) added; Sections 1, 3 and 4 updated for accounts, meal slots, Healthy Levels and prep tiers
0.4	November 11, 2026	GROUP PNP	*Revised design:* • Data model simplified from 8 tables to 5 tables with plural names (users, saved_recipes, ingredients, ingredient_amounts, planned_meals); every table has exactly one primary key and maps to one Java class. • Table 2 updated: shopping list grouped by food group; forgot password and a recipe search bar with Today's recommendations added. • Simpler design: 75 classes reduced to 57; streams instead of loops; Json reads the JSON into a record instead of JsonElement. • New web-style interface with the team logo, text-only recipe cards, animated page changes and an offline message under the search bar; prep time is now estimated and calories are remembered. • Known issues updated (Appendix C).

1. Introduction & Project Overview

Plate N Platter is a personal meal-planning desktop application for students and busy households, built for Option H (Recipe & Meal Planner) of CPE 314. Each user signs up with a security question, logs in, and can reset a forgotten password by answering that question. The Recipes tab opens with a search bar and six Today's recommendations picked for the time of day; nothing else is listed until the user searches TheMealDB by name. When a recipe is chosen, the app estimates its prep time from the recipe's steps and ingredients and fills in the calories the user entered the last time they planned it, so planning takes one click. Every planned meal has a meal slot (Breakfast, Lunch, Dinner or Snack), a Healthy Level (Healthy, Moderate or Indulgent) worked out from its calories and the slot's budget, and a prep tier (Quick, Average or Slow). The app builds a shopping list grouped by food group, keeps a planning streak with a badge, imports past meals from a CSV file in the background, and mines the plan into a grouped bar chart. It solves the everyday problem of deciding what to cook, knowing what to buy, and seeing how healthy one's eating really is, in one simple tool.

Table 2: Project summary

Item	Details
Theme	Recipe & Meal Planner
Target users	Individual users (students, busy households); each user has a private account and sees only their own meals
Platform	Java 17+ desktop application with a JavaFX 17 graphical interface, built with Maven
Data storage	One SQLite file (plate_n_platter.db) with 5 tables, accessed through JDBC; every query uses PreparedStatement
Public API	TheMealDB (https://www.themealdb.com/api.php): search by name, look up by id and filter by category, with the free test key "1"
Key features	• Log in, sign up and forgot password (security question); Account tab • Recipe search bar with Today's recommendations and a message when there is no internet • Prep time estimated automatically; calories remembered from the user's history • Today by meal slot; Meal Plan calendar colored by Healthy Level • Shopping list grouped by food group • Nested recipes (recursion); planning streak and prep-tier badge • Background CSV import; Analytics grouped bar chart
Repository	https://github.com/Kimonooo/cpe314-69167-capstoneteam-Meal_plannerGroup1

2. Preliminary ER Sketch & Team Roles

Figure 1 shows the five tables of our data model, their key attributes and the cardinality of every relationship. Three naming rules make the sketch easy to read:

- **Table names are plural** (users, saved_recipes, ingredients, ingredient_amounts, planned_meals), because each table holds many rows.
- **Exactly one primary key per table**, named after one row of the table: user_id, saved_recipe_id, ingredient_id, ingredient_amount_id and planned_meal_id.
- **A foreign key has the same name as the key it points to**, so saved_recipes.user_id is plainly the user_id of users.

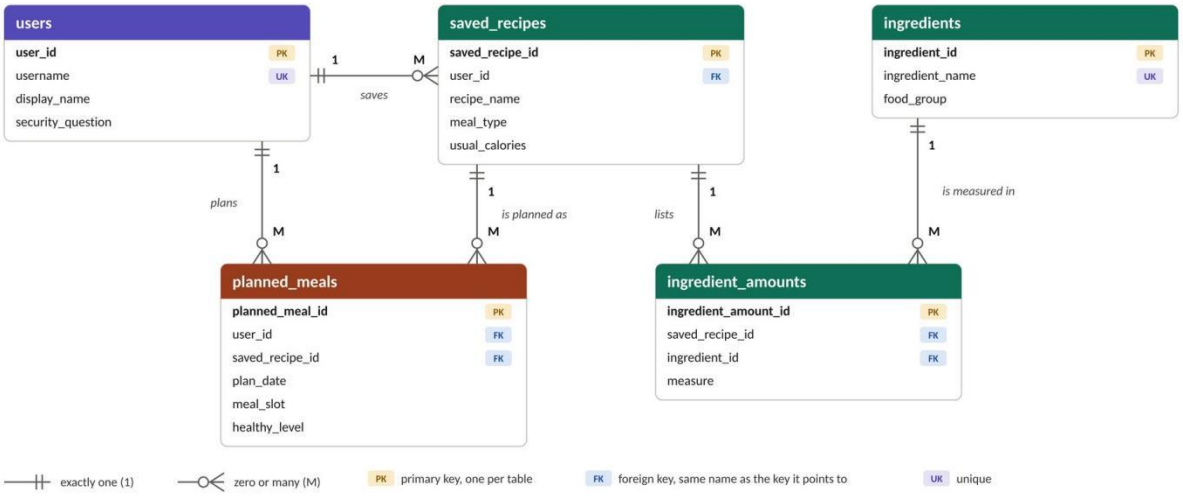


Figure 1: Preliminary ER sketch: 5 tables, one primary key each, key attributes and cardinality

2.1 Key Attributes and Relationships

Table 3 lists every table with its single primary key, its foreign keys and its key attributes; each table is also exactly one Java class (User, SavedRecipe, Ingredient, IngredientAmount and PlannedMeal, see Table 7). Table 4 reads each line of the sketch as a plain sentence with an example.

Table 3: Tables, keys and key attributes

Table	Primary key (one)	Foreign keys	Key attributes
users	user_id	—	username (unique), display_name, security_question
saved_recipes	saved_recipe_id	user_id → users	recipe_name, meal_type,

Table	Primary key (one)	Foreign keys	Key attributes
			usual_calories
ingredients	ingredient_id	—	ingredient_name (unique), food_group
ingredient_amounts	ingredient_amount_id	saved_recipe_id → saved_recipes ingredient_id → ingredients	measure
planned_meals	planned_meal_id	user_id → users saved_recipe_id → saved_recipes	plan_date, meal_slot, healthy_level

Table 4: Relationships in the preliminary sketch

#	One side (1)	Many side (M)	Read the line as	Example
1	users	saved_recipes linked by user_id	One user saves many recipes	Kim saves Chicken Adobo and Spicy Arrabiata Penne
2	users	planned_meals linked by user_id	One user plans many meals	Kim plans three meals on 9 November
3	saved_recipes	planned_meals linked by saved_recipe_id	One saved recipe is planned as many meals	Chicken Adobo is lunch on Monday and dinner on Thursday
4	saved_recipes	ingredient_amounts linked by saved_recipe_id	One saved recipe lists many ingredient amounts	Spicy Arrabiata Penne lists 8 amounts, such as 3 cloves of garlic
5	ingredients	ingredient_amounts linked by ingredient_id	One ingredient is measured in many ingredient amounts	garlic: 3 cloves in the Penne, 1 head in the Adobo

How to read it. A recipe has many ingredients and an ingredient is used in many recipes, which is a many-to-many link. ingredient_amounts is the bridge table that turns it into two simple one-to-many links (rows 4 and 5), and it is also where the amount, such as “3 cloves”, is kept. Every relationship is optional on the many side: a new user has no recipes yet, and a recipe that was never planned has no planned meals.

Revision note (version 0.4). The earlier model had 8 tables and two of them used two-column keys. It is now 5 tables with one key each: recipes became saved_recipes; recipe_ingredients became ingredient_amounts with its own key; meal_plan_entries and meal_history were merged into planned_meals; meal_ingredient was removed because ingredients are now linked by id instead of by recipe name; and recipe_components was removed because nested recipes are only used in memory to

show recursion. Streaks, badges and the shopping list are still worked out from planned_meals, so they need no tables.

2.2 Team Roles

Table 5: Team members and primary roles

Member	Primary role	Owns in the code
Kimberly Luistro	Team Leader	SavedRecipe hierarchy, UndoStack, CookingQueue, DataChangeNotifier; MealDbClient and RecommendationService; PlateNPlatterApp and AppShell (page animation); Analytics page and chart; reviews and merges pull requests
Nesley Cate Librea	User Interface (UI) Developer	Theme and stylesheet; Log in, Sign up, Forgot Password and Account pages; Recipes page (search bar, recommendation cards, offline message); Meal Plan calendar; CategoryMapper
Venice Nadela	Quality Assurance (QA) Tester	JUnit 5 test suite (30 test classes, 173 tests) and the known-issues list
Yvonne Mae Tapan	User Interface (UI) Developer	HeaderBar (logo and navigation pills); Today and Import pages with CsvImportTask; MealRows; StreakCalculator and the prep-tier badge; TextFormat, ChartMath
Kirk Ryan Quisagan	Software Developer	MealSlot, HealthyLevel, FoodGroup; recursive RecipeCalculator; PrepTimeEstimator and CsvRecipeParser; HealthClassifier; StatisticsService; ShoppingList and the Shopping List page
Kenneth Tamidles	Database Administrator	schema.sql (5 tables); User, Ingredient, IngredientAmount, PlannedMeal; DatabaseConnector, UserDao, MealPlanDao; PlanStore, InMemoryPlanStore; AuthService with password reset; PasswordHasher

3. System Architecture & Object-Oriented Design

3.1 Class Hierarchy

Plate N Platter has two class hierarchies (Figure 2): an abstract SavedRecipe with three concrete subclasses, and a PlanStore interface with two implementations. PlateNPlatterApp and RecipeDemo sit in the root package, and the other 55 of the 57 classes are in six packages: model (one class per table, plus the enums), db (all SQL), api (TheMealDB), service (rules), util (small helpers) and ui (JavaFX screens). A package calls the packages below it (ui → service and util → db and api → model); the one exception is the cycle check, which SavedRecipe asks RecipeCalculator for.

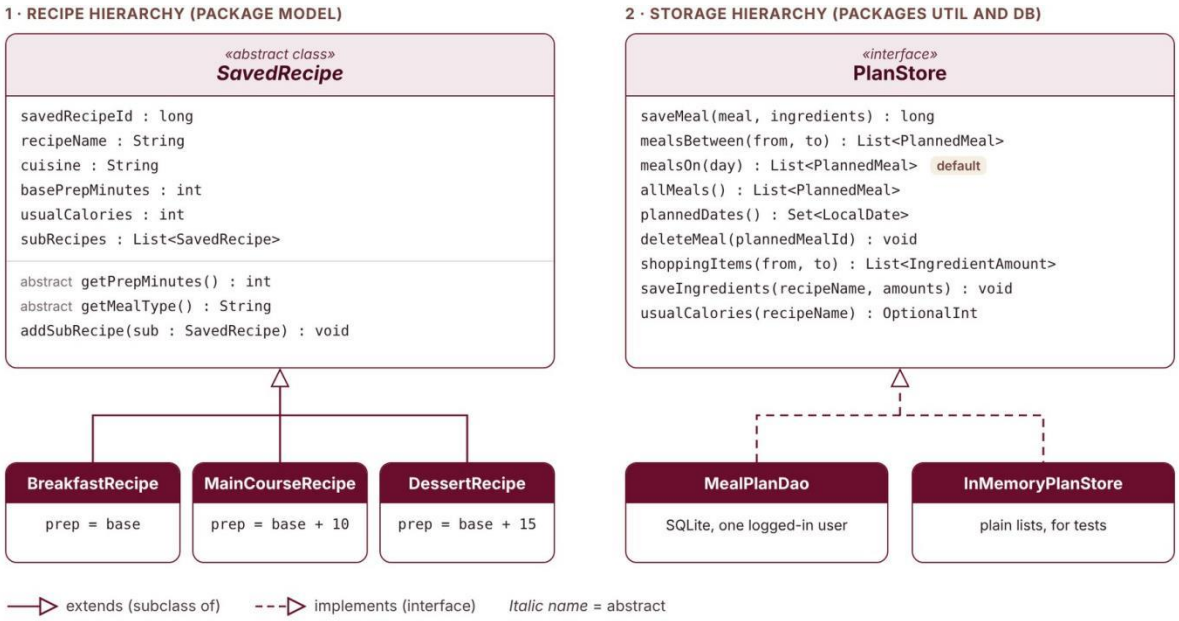


Figure 2: Class hierarchy: abstract SavedRecipe with three subclasses, and the PlanStore interface with two implementations

Code that totals prep time calls getPrepMinutes() on a SavedRecipe reference and never checks which subclass it is, which is polymorphism at runtime. The screens only know PlanStore, so the same screen runs on the SQLite database or on the in-memory test store.

3.2 Abstract Class SavedRecipe and the PlanStore Interface

Abstract class SavedRecipe. SavedRecipe (package model) is one row of saved_recipes. It holds what every recipe has (savedRecipeId, recipeName, cuisine, basePrepMinutes, usualCalories, mealDbId and a list of sub-recipes) and declares getPrepMinutes() and getMealType(). The meal_type column decides which subclass is created, and each subclass adds its own prep-time rule (Table 6).

Table 6: SavedRecipe subclasses and their prep-time rule

Subclass	meal_type	getPrepMinutes()	Reason for the extra time
BreakfastRecipe	BREAKFAST	base	none
MainCourseRecipe	MAIN_COURSE	base + 10	cooking setup
DessertRecipe	DESSERT	base + 15	chilling or setting

```
public abstract class SavedRecipe {
    private final List<SavedRecipe> subRecipes = new ArrayList<>();
    protected SavedRecipe(long savedRecipeId, String recipeName, String cuisine,
            int basePrepMinutes, int usualCalories, String mealDbId) { ... }
    public abstract int getPrepMinutes();   // each subclass adds its own time
    public abstract String getMealType();
    public void addSubRecipe(SavedRecipe sub) {
        if (RecipeCalculator.wouldCreateCycle(this, sub)) {
            throw new CircularRecipeException(this, sub);
        }
        subRecipes.add(sub);
    }
}

public class DessertRecipe extends SavedRecipe {
    @Override public int getPrepMinutes() { return getBasePrepMinutes() + 15; }
    @Override public String getMealType() { return "DESSERT"; }
}
```

Condensed from model/SavedRecipe.java and model/DessertRecipe.java (version 0.4 replaces Recipe with SavedRecipe).

Recursion. RecipeCalculator walks the sub-recipe tree with no loop. The base case is a recipe with no sub-recipes: its stream is empty, so the sum is 0 and only its own time is returned. Otherwise the method calls itself for each sub-recipe, to any depth. A loop in the tree is stopped once, when it would be made: addSubRecipe asks wouldCreateCycle, itself recursive, and throws CircularRecipeException, so the walk can never run forever. In RecipeCalculatorTest, a Cake (Dessert, base 30) holds a Filling (Main Course, base 20), which holds a Cream (Breakfast, base 10): total prep = (30 + 15) + (20 + 10) + 10 = 85 minutes, which is Slow.

```
/** Base case: no sub-recipes, so the stream is empty and its sum is 0. */
public static int totalPrepMinutes(SavedRecipe recipe) {
    return recipe.getPrepMinutes() + recipe.getSubRecipes().stream()
            .mapToInt(RecipeCalculator::totalPrepMinutes)   // recursive case
            .sum();
}

/** True when sub is the parent itself or already contains it at any depth. */
public static boolean wouldCreateCycle(SavedRecipe parent, SavedRecipe sub) {
    return sub == parent || sub.getSubRecipes().stream()
            .anyMatch(inner -> wouldCreateCycle(parent, inner));
}
```

From util/RecipeCalculator.java. totalCalories follows the same pattern.

Interface PlanStore. PlanStore (package util) lists what a screen may ask of saved meals: save a meal with its ingredient amounts, read the meals of a date range or of all time (mealsOn is a default method that calls mealsBetween for one day), list the planned dates, delete a meal, read the ingredient amounts for the shopping list, save the ingredients found online, and read a recipe's usual calories. MealPlanDao implements it with SQLite and is created for one logged-in user, so every query only sees that user's rows. InMemoryPlanStore implements it with plain lists for the tests.

Enums. MealSlot (Breakfast, Lunch, Dinner, Snack, with budgets of 500, 650, 650 and 200 kcal that add up to a 2,000-calorie day), HealthyLevel (Healthy, Moderate, Indulgent), PrepTier (Quick up to 15 min, Average 16 to 45 min, Slow over 45 min) and FoodGroup (the eight groups of the shopping list) . The Healthy Level is stored with each meal; the prep tier is worked out from the minutes whenever it is needed, so it can never disagree with them.

3.3 One Table, One Class

Every table is represented by exactly one class in the model package, and every column by a field with the same name written in camelCase, so a reader always knows where a variable belongs: planned_meals.plan_date is PlannedMeal.planDate (Table 7).

Table 7: Each table and its one Java class

Table	Java class (model)	Fields (column names in camelCase)	Read and written by
users	User	userId, username, displayName, age, gender, securityQuestion. passwordHash, salt and securityAnswerHash stay inside UserDao, so no screen ever sees them.	UserDao
saved_recipes	SavedRecipe (abstract) with BreakfastRecipe, MainCourseRecipe, DessertRecipe	savedRecipeId, recipeName, cuisine, basePrepMinutes, usualCalories, mealDbId; meal_type is given by the subclass; user_id is not repeated, because each MealPlanDao already belongs to one user	MealPlanDao
ingredients	Ingredient	ingredientId, ingredientName, foodGroup (FoodGroup enum)	MealPlanDao
ingredient_amounts	IngredientAmount	ingredientAmountId, savedRecipeId, ingredient (an Ingredient), measure	MealPlanDao
planned_meals	PlannedMeal	plannedMealId, userId, recipe (the SavedRecipe it plans, read with the JOIN), planDate, mealSlot, prepMinutes, calories, healthyLevel	MealPlanDao

A CSV row, a recipe chosen on the Recipes page and a row read from SQLite all become the same PlannedMeal object, so there is no separate class for each source.

One meaning per name. Because "recipe" appears in several places, each name has only one meaning (Table 8). No table or class is called just "recipe" or "recipes".

Table 8: What each recipe-related name means

Name	What it is	Example
saved_recipes	Table: every recipe a user has saved, one row each	Spicy Arrabiata Penne, saved by Kim
SavedRecipe	Java class: one row of saved_recipes	a MainCourseRecipe named Spicy Arrabiata Penne
recipe_name / recipeName	Column and field: the name of a saved recipe	"Spicy Arrabiata Penne"
ingredient_amounts / IngredientAmount	Table and class: one ingredient with its measure in one saved recipe	3 cloves of garlic in the Penne
MealData	Java record: a recipe read from TheMealDB that is not saved yet	a search result for "penne"
Recipes page (RecipesScreen)	The page where the user searches and sees Today's recommendations	Figure 3

3.4 Single Responsibility of Each Class

Table 9 explains how each class in the two hierarchies follows the Single Responsibility Principle: it has one job and one reason to change. Table 10 does the same for the other main classes.

Table 9: Single Responsibility in the two hierarchies

Class	Kind	Its one responsibility	It changes only when
SavedRecipe	Abstract class (model)	Holds the data every saved recipe has and declares getPrepMinutes() and getMealType()	the data every recipe carries changes
BreakfastRecipe	Concrete subclass	The breakfast prep-time rule (base minutes)	the breakfast rule changes
MainCourseRecipe	Concrete subclass	The main-course rule (base + 10 minutes)	the main-course rule changes
DessertRecipe	Concrete subclass	The dessert rule (base + 15 minutes)	the dessert rule changes

Class	Kind	Its one responsibility	It changes only when
PlanStore	Interface (util)	Lists what a screen may ask of saved meals; holds no storage code	screens need a new kind of question
MealPlanDao	Implementation (db)	All SQL for saved_recipes, ingredients, ingredient_amounts and planned_meals, for one user	those tables or their SQL change
InMemoryPlanStore	Implementation (util)	Keeps meals in plain lists for the tests	tests need new fake behavior

Table 10: Other main classes and their single responsibility

Package	Class	Its one job
(root)	PlateNPlatterApp	Starts the app, opens the database, shows Log in, then opens the pages for the logged-in user
ui	AppShell, HeaderBar	AppShell swaps the pages with a short animation; HeaderBar is the top bar with the logo, the navigation pills, the greeting, streak, badge and Log out
ui	Theme	Holds the brand colors and loads platenplatter.css
ui	LoginScreen, RegisterScreen, ForgotPasswordScreen, AccountScreen	Log in, sign up with a security question, reset a password, change the display name
ui	TodayScreen, MealPlanScreen, RecipesScreen, ShoppingListScreen, ImportScreen, AnalyticsScreen	One page each: show data and pass clicks on to the logic classes
ui	CsvImportTask, AnalyticsChart	Imports a CSV file on a background thread; builds the labeled grouped BarChart
service	AuthService	Sign-up, log-in, rename and password-reset rules
service	StatisticsService	Counts and averages for the Analytics tab
service	RecommendationService	Picks the meal slot, the category and six recipes for the time of day
api	MealDbClient	Calls TheMealDB and reads the JSON into MealData records
util	PrepTimeEstimator	Estimates prep minutes from a recipe's steps and ingredient count
util	HealthClassifier, CategoryMapper	The Healthy Level rule; a TheMealDB category to a

Package	Class	Its one job
		meal type
util	RecipeCalculator	Recursive totals and the cycle check for nested recipes
util	CsvRecipeParser	Turns one CSV line into a PlannedMeal, or rejects it with a reason
util	ShoppingList	Groups the ingredient amounts of a date range by food group
util	StreakCalculator	Current and best streak, and the prep-tier badge
util	PasswordHasher, DataChangeNotifier	Salted PBKDF2 hashes; tells every page to refresh after a change (Observer pattern)
db	DatabaseConnector, UserDao	Opens connections with foreign keys on and a 5-second busy timeout; all SQL for users

3.5 Simpler, Smaller Code

Version 0.4 makes the code shorter and easier to follow without losing a feature. Table 11 compares it with version 0.3.

Table 11: What was simplified in version 0.4

Version 0.3	Version 0.4	Why it is simpler
75 classes	57 classes	24 classes removed or merged, 6 added
8 tables, two with two-column keys	5 tables, one id key each	Every row is found and linked by one number
Planned meals in two tables (meal_plan_entries, meal_history) and three classes (MealPlanEntry, ImportRow, PlannedMeal)	One table, planned_meals, and one class, PlannedMeal	One place to read and write a meal
Ingredients matched by recipe-name text (meal_ingredient)	ingredient_amounts linked by ids; each ingredient stored once in ingredients	No spelling mismatches; the food group is set once per ingredient
Shopping list spread over 7 classes	ShoppingList and FoodGroup	One stream groups the list (below); “Look up online” moved into MealDbClient
for and while loops in the logic	Java streams: map, filter, groupingBy, sum	Each method says what it computes, not how to step through it

Version 0.3	Version 0.4	Why it is simpler
JsonElement and JsonObject walked by hand	Gson reads the JSON straight into a record	No manual tree walking or null checks per field
Two prep scales (Quick / Easy / Hard and Quick / Average / Slow)	PrepTier only	One rule everywhere
Cycle guard carried through every recursive walk	Cycle refused once, in addSubRecipe	The total is a three-line recursion
Preview, demo and unused classes (App, CalendarScreen, Database, DatabaseDemo, UserDaoDemo, MealPlanAnalyzer, MealPlanBook, the two old preview tabs, ImportAnalyticsPreview, RecipesScreenPreview, Loader, RowSink)	Removed; RecipeDemo kept to show the stack and the queue	Less code to read and to test
StatFormat, TimeFormat, DateTimeText	TextFormat	All value-to-text helpers in one place

The shopping list shows how much shorter the code becomes: one stream in ShoppingList .build does the work of the seven old classes.

```
/** Food group -> ingredient -> measures, e.g. garlic -> [3 cloves, 1 head]. */
public static Map<FoodGroup, Map<String, List<String>>> build(
        List<IngredientAmount> items) {
    return items.stream().collect(groupingBy(
        item -> item.getIngredient().getFoodGroup(),
        () -> new EnumMap<>(FoodGroup.class),          // groups in fixed order
        groupingBy(item -> item.getIngredient().getIngredientName()
                        .toLowerCase(),
                   TreeMap::new,                        // A to Z in a group
                   mapping(IngredientAmount::getMeasure, toList()))));
}
```

Condensed from util/ShoppingList.java; the real code writes the types out so the Eclipse compiler accepts it.

Performance. The app also does less work:

- No food photos are downloaded (TheMealDB's strMealThumb is ignored), so recipe cards appear as soon as the JSON arrives.
- Today's recommendations are fetched once per part of the day and kept in memory; moving between pages does not call TheMealDB again.
- Each page reads its data with one JOIN query; an index on planned_meals (user_id, plan_date) keeps the calendar, Today and the streak fast.
- A recipe's ingredients are saved once in ingredient_amounts, not again for every planned meal.

- The only loop left in the code is JDBC's while (rs.next()), which reading a ResultSet requires; it is written once in a readAll helper and reused by every query.

3.6 User Interface Design

The interface looks like a modern web page instead of a default JavaFX window (Figure 3). The colors come from the team logo: maroon for buttons and the active page, cream for the background and brown for labels. Table 12 explains how each part is built.

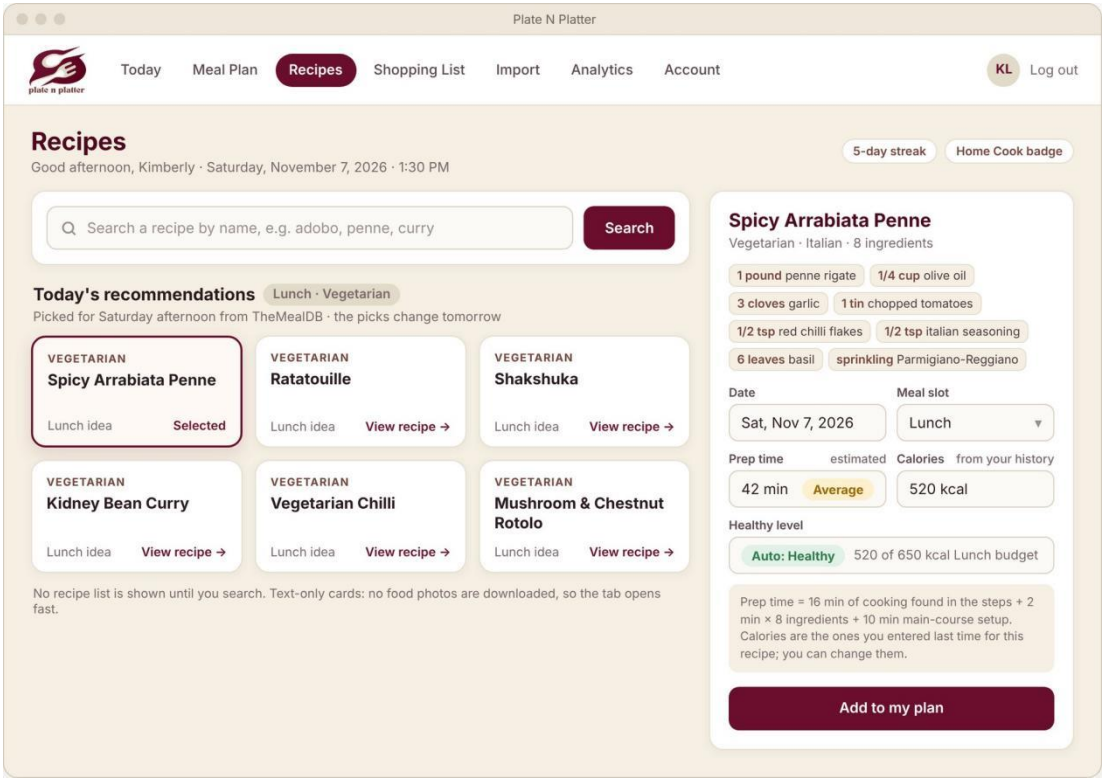


Figure 3: Design of the Recipes page: logo and navigation pills, search bar, Today's recommendations as text-only cards, and the Add to my plan panel

Table 12: How the web-style interface is built in JavaFX

Part	How it is built
Navigation bar	HeaderBar: the logo (an ImageView of logo.png), then seven ToggleButtons in one ToggleGroup styled as rounded pills: Today, Meal Plan, Recipes, Shopping List, Import, Analytics and Account. The selected pill is filled maroon. This replaces the TabPane.
Colors and font	Theme and platenplatter.css: maroon #6B0E2D, cream #F6F0E4 and brown #764640 from the logo; Healthy green, Moderate amber and Indulgent red are the same on every page. Font: Segoe UI, the Windows system font.
Cards	White panes with the .pnp-card style: 14 px rounded corners, a thin border and a soft drop shadow; on hover a recipe card's border turns maroon.

Part	How it is built
Recipe cards	Text only: the category, the recipe name and the meal slot. There is no ImageView and no photo download, which keeps the page clean and fast.
Recipes page	Empty until the user searches, apart from Today's recommendations (Section 5.6). Search results replace the picks; Clear brings them back.
Page change	AppShell fades the new page in while sliding it up 16 px, in 250 ms (code below and Figure 4).
No internet	A red message with a Retry button appears under the search bar (Section 5.7).

Page transition. Every click on a navigation pill calls AppShell.show. A FadeTransition and a TranslateTransition run together in a ParallelTransition, and EASE_OUT makes the movement start fast and stop softly.

```
private void show(Node page) {
    content.getChildren().setAll(page);                    // swap the page in
    Duration time = Duration.millis(250);
    FadeTransition fade = new FadeTransition(time, page);
    fade.setFromValue(0);                                  // invisible ...
    fade.setToValue(1);                                    // ... to fully shown
    TranslateTransition slide = new TranslateTransition(time, page);
    slide.setFromY(16);                                    // 16 px lower ...
    slide.setToY(0);                                       // ... to its place
    slide.setInterpolator(Interpolator.EASE_OUT);          // fast start, soft stop
    new ParallelTransition(fade, slide).play();
}
```

From ui/AppShell.java.



Figure 4: The page change after clicking Recipes, at 0, 125 and 250 ms

3.7 Forgot Password Flow

Sign up now asks the user to pick one of four security questions (the name of their first pet, the city they were born in, their favorite food, or their elementary school) and to type the answer. A "Forgot password?" link on the Log in page opens ForgotPasswordScreen:

1. The user types their username. A background Task reads the security question for that username.
2. The page shows the question. The user types the answer, a new password and the new password again.
3. AuthService.resetPassword checks the answer and the password rules, then saves a new password hash. The page returns to Log in with “Password changed. You can log in now.”

```
public void resetPassword(String username, String answer, String newPassword,
        String confirmPassword) throws AuthException, SQLException {
    User user = users.findByUsername(trim(username)).orElseThrow(
            () -> new AuthException("No account with that username."));
    checkPassword(newPassword, confirmPassword);      // 6+ characters, typed twice
    if (!users.answerMatches(user.getUserId(), normalize(answer))) {
        throw new AuthException("That answer does not match.");
    }
    users.updatePassword(user.getUserId(), newPassword);   // new PBKDF2 hash
}

public static String normalize(String answer) {      // " Bantay " = "bantay"
    return answer == null ? "" : answer.trim().toLowerCase(Locale.ROOT);
}
```

From service/AuthService.java.

The answer is never stored as text: like the password, it is saved as a salted PBKDF2-HMAC-SHA256 hash (65,536 rounds) in users.security_answer_hash. The answer is trimmed and changed to lower cas e before hashing, so “ Bantay ” and “bantay” are the same answer.

3.8 Design Rationale

SavedRecipe is abstract because there is no plain recipe: every recipe is one of three types, and the type changes how long it takes. PlanStore is an interface so the screens depend on what they need, not on SQLite, and the tests swap in InMemoryPlanStore with no database file. One class per table means a change to a table touches exactly one model class and one DAO. The screens never write SQL and the DAOs never touch JavaFX, so a change to the database or to the look stays inside one package, and if TheMealDB changes its JSON, only MealDbClient changes.

4. Data Model & Database Design

4.1 Database Overview

The final database is one SQLite file, plate_n_platter.db, with the 5 tables defined in src/main/resources/schema.sql (Figure 5). They are the same 5 tables as the sketch in Figure 1 and Table 3, now with every column and its type. All five are used by the app: users for accounts, saved_recipes and planned_meals for every page, and ingredients with ingredient_amounts for the recipe details and the shopping list.

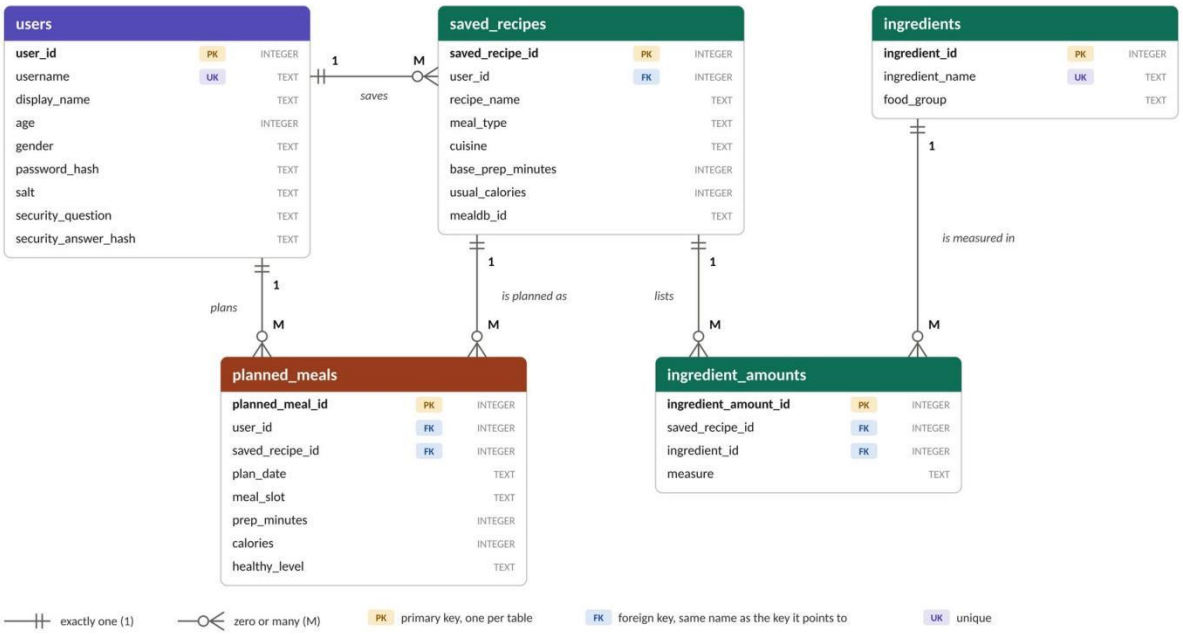


Figure 5: Final ER diagram of plate_n_platter.db (UNIQUE and CHECK rules are in Table 13)

4.2 Tables and Relationships

Table 13: The 5 tables in schema.sql

Table	Columns	Keys and rules	Used by
users	user_id, username, display_name, age, gender, password_hash, salt, security_question, security_answer_hash	PK user_id; username UNIQUE (case does not matter); hashes and salt NOT NULL	Log in, Sign up, Forgot Password, Account
saved_recipes	saved_recipe_id, user_id, recipe_name, meal_type,	PK saved_recipe_id; FK user_id; UNIQUE (user_id,	Recipes, every meal list

Table	Columns	Keys and rules	Used by
	cuisine, base_prep_minutes, usual_calories, mealdb_id	recipe_name); meal_type is BREAKFAST, MAIN_COURSE or DESSERT	
ingredients	ingredient_id, ingredient_name, food_group	PK ingredient_id; ingredient_name UNIQUE (case does not matter)	Recipes, Shopping List
ingredient_amounts	ingredient_amount_id, saved_recipe_id, ingredient_id, measure	PK ingredient_amount_id; FK saved_recipe_id and ingredient_id; UNIQUE (saved_recipe_id, ingredient_id)	Recipes, Shopping List
planned_meals	planned_meal_id, user_id, saved_recipe_id, plan_date, meal_slot, prep_minutes, calories, healthy_level	PK planned_meal_id; FK user_id and saved_recipe_id; CHECK rules on meal_slot, healthy_level, prep_minutes and calories	Today, Meal Plan, Shopping List, Analytics, streak

Table 14: Relationships in the final schema and what happens on delete

Relationship (one → many)	Foreign key column	Cardinality	When the "one" row is deleted
users → saved_recipes	user_id	1 : M	The user's saved recipes are deleted too (ON DELETE CASCADE)
users → planned_meals	user_id	1 : M	The user's planned meals are deleted too (ON DELETE CASCADE)
saved_recipes → planned_meals	saved_recipe_id	1 : M	Refused while the recipe is still planned
saved_recipes → ingredient_amounts	saved_recipe_id	1 : M	The recipe's amounts are deleted too (ON DELETE CASCADE)
ingredients → ingredient_amounts	ingredient_id	1 : M	Refused while a recipe still uses the ingredient

4.3 Core Table Schema

planned_meals is the table every page reads; the other four CREATE TABLE statements are in schema.sql.

```
CREATE TABLE IF NOT EXISTS planned_meals (
    planned_meal_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id         INTEGER NOT NULL
                    REFERENCES users(user_id) ON DELETE CASCADE,
    saved_recipe_id INTEGER NOT NULL
                    REFERENCES saved_recipes(saved_recipe_id),
    plan_date       TEXT    NOT NULL,               -- yyyy-mm-dd
    meal_slot       TEXT    NOT NULL
                    CHECK (meal_slot IN ('BREAKFAST', 'LUNCH', 'DINNER', 'SNACK')),
    prep_minutes    INTEGER NOT NULL CHECK (prep_minutes >= 0),
    calories        INTEGER NOT NULL DEFAULT 0 CHECK (calories >= 0),
    healthy_level   TEXT    NOT NULL
                    CHECK (healthy_level IN ('HEALTHY', 'MODERATE', 'INDULGENT'))
);
CREATE INDEX IF NOT EXISTS idx_planned_meals_user_date
    ON planned_meals (user_id, plan_date);
```

From src/main/resources/schema.sql.

- **One primary key per table.** Each table has one INTEGER key; pairs that must not repeat, such as a recipe and an ingredient, are protected with UNIQUE instead of a two-column key.
- **Rules live in the database too.** meal_slot, meal_type and healthy_level only accept their enum names, and minutes and calories cannot be negative, so SQLite rejects a bad row even if a Java check is skipped.
- **Foreign keys are on.** DatabaseConnector runs PRAGMA foreign_keys = ON for every connection, so a meal cannot point to a missing user or recipe.
- **Passwords and answers are never stored.** users keeps a salted PBKDF2-HMAC-SHA256 hash of the password and of the security answer; one random salt per user is used for both.
- **A meal keeps its own numbers.** saved_recipes holds the usual values (base_prep_minutes, usual_calories); planned_meals keeps the minutes and calories of that one meal, so editing a recipe never changes past meals.
- **user_id is also on planned_meals** so the calendar and the streak read one user's dates through one index without a join. MealPlanDao always writes its own user id to both tables, so they cannot disagree.

4.4 Query Design

Every query uses PreparedStatement and is scoped to one user: the MealPlanDao made at log-in holds that user's id and binds it as the first parameter. Table 15 lists the main queries; the snippet shows the one behind Today, the calendar and the chart.

Table 15: Main queries in MealPlanDao

Method	Tables joined	Used by
mealsBetween(from, to), allMeals()	planned_meals JOIN saved_recipes	Today, Meal Plan, Analytics, streak
shoppingItems(from, to)	planned_meals JOIN ingredient_amounts	Shopping List

Method	Tables joined	Used by
	JOIN ingredients	
saveMeal(meal, amounts)	INSERT into saved_recipes (once per name), ingredients (once per name), ingredient_amounts and planned_meals, in one transaction	Recipes, CSV import
usualCalories(recipeName)	saved_recipes	Recipes (calories filled in)

```
static final String MEALS_BETWEEN = """
    SELECT p.planned_meal_id, p.plan_date, p.meal_slot, p.prep_minutes,
           p.calories, p.healthy_level, r.saved_recipe_id, r.recipe_name,
           r.meal_type, r.cuisine, r.base_prep_minutes, r.usual_calories
    FROM planned_meals p
    JOIN saved_recipes r ON r.saved_recipe_id = p.saved_recipe_id
    WHERE p.user_id = ? AND p.plan_date BETWEEN ? AND ?
    ORDER BY p.plan_date, p.planned_meal_id""";

public synchronized List<PlannedMeal> mealsBetween(LocalDate from, LocalDate to)
        throws SQLException {
    try (PreparedStatement ps = conn.prepareStatement(MEALS_BETWEEN)) {
        ps.setLong(1, userId);                    // only this user's meals
        ps.setString(2, from.toString());
        ps.setString(3, to.toString());
        return readAll(ps, MealPlanDao::toPlannedMeal);
    }
}
```

Condensed from db/MealPlanDao.java (it replaces MealPlanHistoryDao in version 0.4).

4.5 Agreement with the ER Sketch

Figure 1, Figure 5, Table 3 and Table 13 list exactly the same 5 tables and keys, and Table 4 and Table 14 list the same 5 relationships. Compared with the first sketch in version 0.1, the changes are:

- Streaks and badges are worked out from the dates in planned_meals, so there are no Streak or UserBadge tables.
- The shopping list is built from ingredient_amounts for a date range when it is opened, so there is no ShoppingListItem table.
- The cook queue and undo stack live in memory (CookingQueue, UndoStack), and nested recipes are only built in memory, so they have no tables. Favorites, a stretch goal, was dropped.
- The earlier tables meal_plan_entries, meal_history, meal_ingredient and recipe_components were merged or removed (Section 2.1), leaving one table per thing the app stores.

5. API Integration Design

5.1 The TheMealDB Calls

Plate N Platter consumes TheMealDB, a free public REST API. MealDbClient (package api) makes three kinds of call and turns the JSON into MealData records (Table 16).

Table 16: The TheMealDB calls

Method	Endpoint (base: https://www.themealdb.com/api/json/v1/1/)	Used for
searchByName(name)	search.php?s={name} (URL-encoded)	The search bar on Recipes; findExact uses it for "Look up online" on the Shopping List
listByCategory(category)	filter.php?c={category}	Today's recommendations (returns only idMeal, strMeal and a photo link we ignore)
lookupById(id)	lookup.php?i={idMeal}	The full recipe after a recommendation card is clicked

All three use the shared free test key 1, which is part of the path, so no sign-up or login is needed. The Java HttpClient sends the header User-Agent: PlateNPlatter/1.0, gives up connecting after 5 seconds and waits at most 10 seconds for an answer. Only HTTP 200 counts as success.

5.2 JSON Response

The response is one object with a meals array. Each meal is a flat object of strings: idMeal, strMeal, strCategory, strArea, strInstructions, twenty numbered slots strIngredient1 to strIngredient20 and their strMeasure1 to strMeasure20. Unused slots are empty strings or null. When nothing matches, meals is null, not an empty array.

```
{
  "meals": [
    {
      "idMeal": "52771",
      "strMeal": "Spicy Arrabiata Penne",
      "strCategory": "Vegetarian",
      "strArea": "Italian",
      "strInstructions": "Bring a large pot of water to a boil...",
      "strIngredient1": "penne rigate",
      "strMeasure1": "1 pound",
      "strIngredient9": "",
      "strIngredient16": null
    }
  ]
}
```

Trimmed: a real meal has all twenty ingredient and measure slots.

5.3 Parsing

MealDbClient.parseMeals lets Gson read the JSON straight into a small record, MealDbResponse, whose meals field is a list of maps from field name to text. No JsonElement or JsonObject is used and no field is read by hand. parseMeals is public and static, so the tests run it on saved JSON without internet.

```
private record MealDbResponse(List<Map<String, String>> meals) { }
private static final Gson GSON = new Gson();

public static List<MealData> parseMeals(String json) {
    MealDbResponse response = GSON.fromJson(json, MealDbResponse.class);
    if (response == null || response.meals() == null) {
        return List.of();                    // {"meals":null}: nothing matched
    }
    return response.meals().stream().map(MealDbClient::toMealData).toList();
}

private static MealData toMealData(Map<String, String> m) {
    List<IngredientAmount> amounts = IntStream.rangeClosed(1, 20)
        .filter(i -> !text(m, "strIngredient" + i).isBlank())   // skip blanks
        .mapToObj(i -> new IngredientAmount(text(m, "strIngredient" + i),
                                            text(m, "strMeasure" + i)))
        .toList();
    return new MealData(text(m, "idMeal"), text(m, "strMeal"),
            text(m, "strCategory"), text(m, "strArea"),
            text(m, "strInstructions"), amounts);
}
```

From api/MealDbClient.java; text(m, key) returns the trimmed value, or "" when it is missing or null.

5.4 Use in the App

Table 17: How the parsed data is used in the app

Field	How the app uses it
idMeal	Saved as saved_recipes.mealdb_id; lookupById uses it to open a recommended recipe

Field	How the app uses it
strMeal	The card title on Recipes; saved as saved_recipes.recipe_name
strCategory	Shown on the card. CategoryMapper turns it into the meal type (Breakfast, Main Course or Dessert), which picks the SavedRecipe subclass and the default meal slot
strArea	Saved as saved_recipes.cuisine and shown under the recipe name
strInstructions	The recipe steps on Recipes; PrepTimeEstimator reads the times written in them (Section 5.5)
strIngredient1 to 20 with strMeasure1 to 20	Saved as ingredient_amounts; each new ingredient name is added once to ingredients with its food group from FoodGroup.of(name)
strMealThumb	Not used: recipe cards are text only

5.5 Prep Time and Calories

TheMealDB gives no prep time and no calories, so the app fills both in and the user only checks them. Prep time is estimated by PrepTimeEstimator in five steps (Table 18).

Table 18: How the prep time is estimated, with Spicy Arrabiata Penne

Step	Rule	Spicy Arrabiata Penne
1. Cooking time	Add every "N minutes" or "N hours" written in the steps; a range such as "1 to 2" counts its second number; hours count 60 minutes	"about 9 minutes" + "1 to 2 minutes" + "for 5 minutes" = 16
2. Handling time	2 minutes for each ingredient (washing, cutting, measuring)	8 ingredients × 2 = 16
3. Base prep	Step 1 + step 2, kept between 5 and 480 minutes; saved as base_prep_minutes	16 + 16 = 32
4. Meal-type extra	getPrepMinutes() of the subclass adds 0, 10 or 15	Vegetarian → Main Course: 32 + 10 = 42 min
5. Prep tier	PrepTier: Quick up to 15, Average 16 to 45, Slow over 45	42 min is Average

```
private static final Pattern TIME = Pattern.compile(
    "(\\d+)(?:\\s*(?:-|to)\\s*(\\d+))?\\s*(hours?|hrs?|minutes?|mins?)\\b",
    Pattern.CASE_INSENSITIVE);

public static int estimateBaseMinutes(String instructions, int ingredientCount) {
    int cooking = TIME.matcher(instructions == null ? "" : instructions)
        .results()                                  // every time phrase found
        .mapToInt(PrepTimeEstimator::minutesOf)
        .sum();
    int total = cooking + MINUTES_PER_INGREDIENT * Math.max(ingredientCount, 0);
    return Math.max(MIN_MINUTES, Math.min(MAX_MINUTES, total));   // 5 min to 8 h
}

private static int minutesOf(MatchResult m) {
    int n = Integer.parseInt(m.group(2) != null ? m.group(2) : m.group(1));
    return m.group(3).toLowerCase(Locale.ROOT).startsWith("h") ? n * 60 : n;
}
```

From util/PrepTimeEstimator.java (MINUTES_PER_INGREDIENT = 2, MIN_MINUTES = 5, MAX_MINUTES = 480).

Calories are remembered. The first time a recipe is planned, the user types its calories. They are saved on the meal and as saved_recipes.usual_calories. The next time the same recipe is planned, the Calories box is already filled in and marked “from your history”; a CSV import updates the same column. The Auto Healthy Level is then worked out right away from the calories and the meal slot’s budget, for example 520 kcal against the 650 kcal Lunch budget is Healthy. The user can still change any of the three values before pressing Add to my plan.

5.6 Today’s Recommendations

When the Recipes page opens, RecommendationService picks a meal slot and a TheMealDB category from the clock (Table 19). The category changes with the day of the week, so the picks do not repeat every day. listByCategory then fetches that category, and six recipes are chosen with a shuffle seeded by the date: the same six all day, new ones tomorrow.

Table 19: What Today’s recommendations show

Time of day	Meal slot	TheMealDB category (rotates by weekday)
Before 11:00	Breakfast	Breakfast
11:00 to 15:59	Lunch	Chicken, Seafood, Vegetarian or Pasta
16:00 to 17:59	Snack	Starter or Side
18:00 onward	Dinner	Beef, Chicken, Seafood, Vegetarian, Pork, Pasta or Lamb

```
private static final List<String> LUNCH = List.of("Chicken", "Seafood",
        "Vegetarian", "Pasta");
private static final List<String> SNACK = List.of("Starter", "Side");
private static final List<String> DINNER = List.of("Beef", "Chicken", "Seafood",
        "Vegetarian", "Pork", "Pasta", "Lamb");

public static Pick pickFor(LocalDateTime now) {
    int hour = now.getHour();
    int day = now.getDayOfWeek().getValue();     // Monday 1 ... Sunday 7
    if (hour < 11) return new Pick(MealSlot.BREAKFAST, "Breakfast");
    if (hour < 16) return new Pick(MealSlot.LUNCH, LUNCH.get(day % LUNCH.size()));
    if (hour < 18) return new Pick(MealSlot.SNACK, SNACK.get(day % SNACK.size()));
    return new Pick(MealSlot.DINNER, DINNER.get(day % DINNER.size()));
}

public static List<MealData> todaysPicks(List<MealData> meals, LocalDate today) {
    List<MealData> copy = new ArrayList<>(meals);
    Collections.shuffle(copy, new Random(today.toEpochDay()));   // same all day
    return copy.stream().limit(6).toList();
}
```

From service/RecommendationService.java. On Saturday, 7 November 2026 at 1:30 PM: day 6, 6 % 4 = 2, so Lunch · Vegetarian, as in Figure 3.

Clicking a card calls lookupById in a background Task to get the full recipe; the Add to my plan panel then shows its ingredients, the estimated prep time, the remembered calories and the meal slot from the pick.

5.7 Error Handling and the Offline Message

When the computer has no internet, a red message appears right under the search bar (Figure 6). It also replaces the recommendation cards, so the page never looks empty without a reason.



Figure 6: The search bar when the computer is offline

```
task.setOnFailed(e -> {                          // the search Task of RecipesScreen
    searchButton.setDisable(false);
    showError(task.getException(), "");
});

private void showError(Throwable error, String listText) {
    bannerText.setText(MealDbClient.isOffline(error) ? OFFLINE : NOT_RESPONDING);
    Theme.show(banner, true);                     // the red message with Retry
    cardGrid.getChildren().clear();
    listStatus.setText(listText);
}

public static boolean isOffline(Throwable error) {         // in MealDbClient
    return Stream.iterate(error, Objects::nonNull, Throwable::getCause)
            .anyMatch(e -> e instanceof ConnectException
                        || e instanceof UnknownHostException
                        || e instanceof HttpConnectTimeoutException);
}
```

From ui/RecipesScreen.java and api/MealDbClient.java. OFFLINE and NOT_RESPONDING are the two messages below; isOffline also looks at the cause of an error.

- **No internet** (the connection is refused, the name cannot be found, or connecting takes over 5 seconds): “No internet connection. Check your Wi-Fi, then press Retry.” Retry repeats the last search or reloads the picks. Saved meals, the calendar and the shopping list keep working.
- **TheMealDB fails** (any status other than 200, or no answer in 10 seconds): “TheMealDB is not responding right now. Try again in a minute.”
- **No match:** "meals": null becomes an empty list, and the page says “No recipes found for ‘…’. Try another word, such as chicken or curry.”
- **Missing fields:** a missing or null field becomes empty text, blank ingredient slots are skipped, and a blank category falls back to Main Course.
- **Wrong dish:** “Look up online” on the Shopping List saves ingredients only for an exact name match (findExact), so a different dish’s ingredients are never attached.
- **Double clicks:** the Search button is disabled while a search runs; pressing Enter in the search bar also searches.

To prove the call is live during the demo, turn Wi-Fi off and search (the red message appears), turn it back on and press Retry (real results), and open the same URL in a browser to show the raw JSON.

6. Concurrency Design

6.1 Long-Running Operations

Anything that can take more than a moment runs off the JavaFX Application Thread: reading a big file, calling the internet, querying SQLite, and hashing a password. Table 20 lists every such operation and what the user sees while it runs. The largest one is the CSV import: the sample files sample_data/meal_history_3000.csv and meal_history_30000.csv hold 3,000 and 30,000 rows.

Table 20: Operations that run on a background thread

Operation	Where	Work done in call()	What the user sees
Import a meal-history CSV	CsvImportTask, started by ImportScreen	Read the lines, parse each into a PlannedMeal, save it with PlanStore.saveMeal	Progress bar and "Imported n rows, skipped k" every 50 rows; Cancel button
Search TheMealDB	RecipesScreen	MealDbClient.searchByName: HTTP call and JSON parsing	"Searching…", Search disabled, then results, "No recipes found" or the offline message
Load Today's recommendations	RecipesScreen, RecommendationService	listByCategory, then todaysPicks	Six cards when ready, or the offline message
Open a recommended recipe	RecipesScreen	lookupById, PrepTimeEstimator and usualCalories	The Add to my plan panel fills in
Save a planned meal	RecipesScreen	PlanStore.saveMeal with its ingredient amounts	Add button disabled until saved; other pages refresh
Log in, sign up, reset password, rename	LoginScreen, RegisterScreen, ForgotPasswordScreen, AccountScreen	AuthService: PBKDF2 hashing (65,536 rounds) and SQL	Button disabled so it cannot run twice; status message
Load Today, the calendar and the streak	TodayScreen, MealPlanScreen, HeaderBar	mealsOn, mealsBetween, plannedDates, StreakCalculator	The data appears when ready
Build the shopping list	ShoppingListScreen	shoppingItems, then ShoppingList.build	The list appears when ready
Mine the data for the chart	AnalyticsScreen	mealsBetween or allMeals, then StatisticsService	"Loading…", Refresh disabled, then the

Operation	Where	Work done in call()	What the user sees
			chart

6.2 Threading Approach

Each operation is a javafx.concurrent.Task that AppShell.runInBackground starts on its own daemon Thread (new Thread(task), setDaemon(true), start()). A daemon thread never keeps the app alive after the window closes. The rule the code follows is simple: call() does the slow work and never touches the screen, while setOnSucceeded, setOnFailed and setOnCancelled run back on the JavaFX Application Thread and update the screen. Figure 7 shows the CSV import end to end.

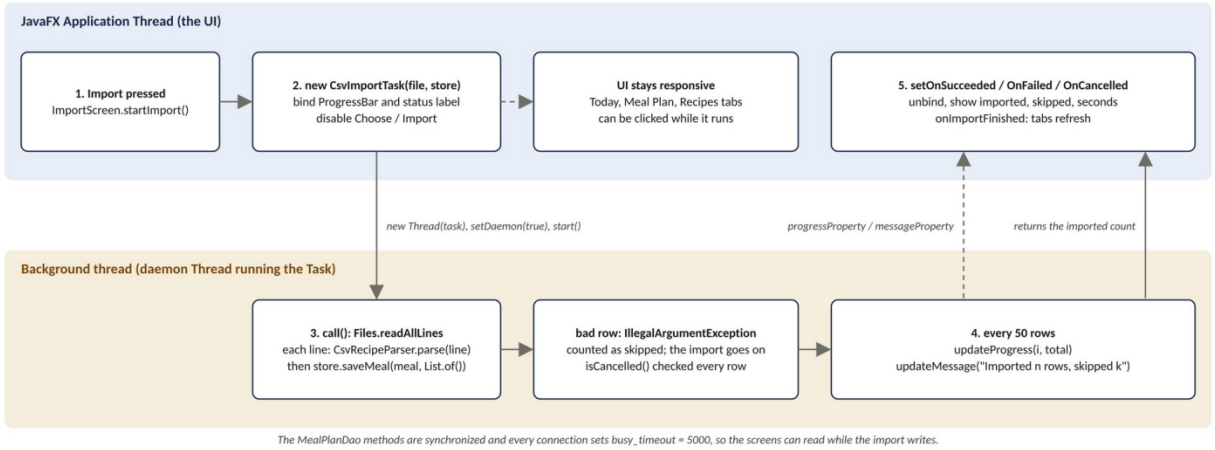


Figure 7: The CSV import: the UI thread starts the Task and stays free while the background thread reads the file

```
CsvImportTask task = new CsvImportTask(chosenFile, store);
bar.progressProperty().bind(task.progressProperty());   // the bar follows the Task
statusLabel.textProperty().bind(task.messageProperty());
task.setOnSucceeded(e -> finish(task, "Done! The import finished.",
        1.0, Theme.Kind.OK, true));
task.setOnFailed(e -> finish(task, "Import failed: "
        + task.getException().getMessage(), 0, Theme.Kind.ERROR, false));
task.setOnCancelled(e -> finish(task, "Import cancelled.",
        0, Theme.Kind.INFO, true));
AppShell.runInBackground(task);          // BACKGROUND THREAD: UI stays responsive

public static void runInBackground(Task<?> task) {          // in AppShell
    Thread thread = new Thread(task);
    thread.setDaemon(true);
    thread.start();
}
```

Condensed from ui/ImportScreen.java (startImport) and ui/AppShell.java.

6.3 How the UI Stays Responsive

- The window never waits: while a 30,000-row file is imported, the user can open Today, Meal Plan or Recipes, the page animation still plays, and the clock in the header keeps ticking.

- Progress is reported, not pushed row by row: CsvImportTask calls updateProgress and updateMessage only every 50 rows, so the UI thread is not flooded.
- Bad rows do not stop the import: CsvRecipeParser throws IllegalArgumentException for a bad line, the Task counts it as skipped and carries on. Cancel works because isCancelled() is checked before each row.
- Buttons that start work are disabled while it runs, so a double click cannot start it twice.
- When an import or a save finishes, DataChangeNotifier.fire() is called on the UI thread and every page refreshes (Observer pattern).

6.4 Thread Safety

- The MealPlanDao methods are synchronized, because the import writes while the pages read through the same connection.
- Every connection sets PRAGMA busy_timeout = 5000, so a second connection waits up to 5 seconds instead of failing with “database is locked”.
- Screen objects are only changed in setOnSucceeded and setOnFailed, never inside call(). The cache of recommendations is a ConcurrentHashMap, so the background thread can fill it safely.

7. Data Mining & Visualization Design

7.1 Aggregation Step

The Analytics tab mines the user's own meals from SQLite. A background Task reads the meals of the chosen week (mealsBetween, Monday to Sunday) or of all time (allMeals). StatisticsService then aggregates them with streams (Table 21). countByDay keeps the meals of that week, groups them by day and then by group with Collectors.groupingBy and counting(), and finally lists all seven days so an empty day still shows zeros. The user picks what the bars are split by: Healthy Level, prep tier or meal slot.

Table 21: Data-mining methods in StatisticsService

Method	What it computes	Shown as
countByDay(meals, monday, grouping)	For one week: day → (group → number of meals); meals outside the week are ignored	The grouped bar chart (Week view)
countByWeekday(meals, grouping)	For all time: Mon…Sun → (group → number of meals)	The grouped bar chart (All time view)
totals(meals, grouping)	Number of meals per Healthy Level, prep tier and meal slot	The breakdown panel with percentages
healthyShare(meals)	Healthy meals ÷ all meals (NaN when there are none)	The "Healthy share" card
averageCalories(meals), averagePrepMinutes(meals)	Averages; calories only over meals whose calories are known	The "Average calories" and "Average prep time" cards
levelByDay(meals)	The level most of a day's meals have; a tie goes to the less healthy level	The color of each day on the Meal Plan calendar

```
public static Map<String, Map<String, Long>> countByDay(
        List<PlannedMeal> meals, LocalDate monday, Grouping grouping) {
    LocalDate sunday = monday.plusDays(6);
    Map<LocalDate, Map<String, Long>> counted = meals.stream()
        .filter(m -> !m.getPlanDate().isBefore(monday)
                  && !m.getPlanDate().isAfter(sunday))          // this week only
        .collect(groupingBy(PlannedMeal::getPlanDate,
                 groupingBy(grouping::seriesOf, counting())));
    return IntStream.range(0, 7).mapToObj(monday::plusDays)     // all 7 days
        .collect(toMap(TextFormat::dayLabel,
                 day -> grouping.withZeros(counted.get(day)),   // empty day: 0s
                 (a, b) -> a, LinkedHashMap::new));
}
```

From service/StatisticsService.java.

7.2 Worked Example

Importing sample_data/meal_history_demo.csv (48 meals over 2 to 15 November 2026) and opening the week of 2 November gives the counts in Table 22: 24 meals, of which 14 are Healthy (a healthy share of 58%), an average of 532.5 kcal (shown on the card as 533 kcal) and 37.5 minutes of prep per meal.

Table 22: Mined counts for the week of 2 November 2026 (Healthy Level grouping)

Group	Mon 2	Tue 3	Wed 4	Thu 5	Fri 6	Sat 7	Sun 8	Total
Healthy	2	3	2	1	2	2	2	14
Moderate	1	1	0	2	1	0	1	6
Indulgent	0	0	1	0	1	1	1	4
All meals	3	4	3	3	4	3	4	24

7.3 Chart and Why It Was Chosen

AnalyticsChart.buildGrouped draws one series per group, with one bar per day: the title “Meals Planned per Day by …” with the week or “All Time”, the x-axis labeled “Day”, the y-axis labeled “Number of meals planned”, and a legend. Healthy is always green, Moderate yellow and Indulgent red, the same colors as the calendar and the meal chips. Figure 8 shows the chart for the counts in Table 22.

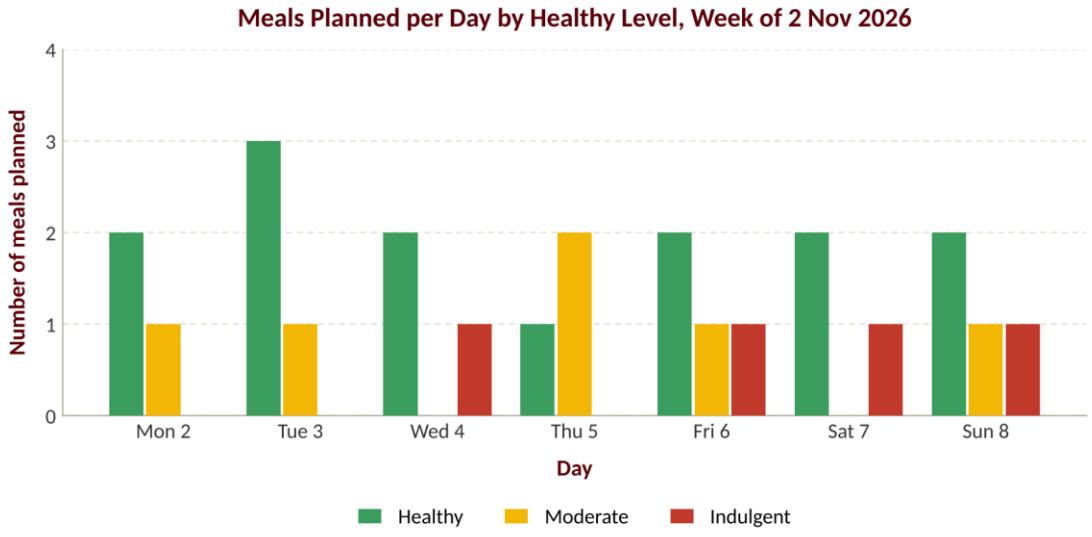


Figure 8: Grouped bar chart of the mined counts for the week of 2 November 2026

Why a grouped bar chart? Days and groups are categories and the counts are small whole numbers, so bars compare them best, and placing the groups side by side shows each day’s mix at a glance. A line chart would suggest smooth change between days, and a pie chart shows only one total, so a week would need seven pies.

The result is not hard-coded. Removing or adding a meal and pressing Refresh changes the bars. StatisticsServiceTest changes one meal's level and EndToEndPipelineTest adds two meals to the database, and both check that the mined counts change with them.

8. Testing Summary

Summarize your JUnit test suite — how many tests exist, what they cover, and at least one meaningful edge case per tested class. A short table or list of key test cases is helpful here.

9. Individual Contributions

List what each team member specifically built or owned, referencing their git commit history, so the instructor can cross-check contribution during the Individual Q&A.

Appendix

A. References and API Documentation

- TheMealDB API documentation (search, lookup and filter endpoints): https://www.themealdb.com/api.php
- Project repository: https://github.com/Kimonooo/cpe314-69167-capstoneteam-Meal_plannerGroup1
- OpenJFX (JavaFX) documentation, including animation: https://openjfx.io/
- SQLite documentation (foreign keys, CHECK, indexes, PRAGMA): https://www.sqlite.org/docs.html
- Xerial SQLite JDBC driver: https://github.com/xerial/sqlite-jdbc
- Gson JSON library (records supported since 2.10): https://github.com/google/gson
- JUnit 5 User Guide: https://junit.org/junit5/docs/current/user-guide/

B. Licensing Notes

- TheMealDB: the test key 1 may be used during development or for educational use; a supporter key is required to release an app publicly on an app store. Recipe text and data belong to TheMealDB; the app does not download or show its food photos.
- Libraries: OpenJFX 17 (GPL v2 with the Classpath Exception), sqlite-jdbc 3.45 (Apache License 2.0), Gson 2.10 (Apache License 2.0), JUnit 5.10 (Eclipse Public License 2.0).
- The Plate N Platter logo was made by the team. The sample CSV files in sample_data/ were made by the team for testing and the demo.

C. Known Issues and Limitations

Table 25: Known issues and limitations

#	Known issue or limit	Where
1	A planned meal cannot be edited. To change it, remove it and add it again.	MealPlanScreen
2	Importing the same CSV twice adds the meals twice (no duplicate check).	CsvImportTask
3	TheMealDB gives no calories, so the user types them the first time a recipe is planned. The Auto level is a simple budget rule, not nutrition advice.	RecipesScreen, HealthClassifier
4	The prep time is an estimate from the text of the steps. Recipes that write no times only get the 2-minutes-per-ingredient part, and “1/2	PrepTimeEstimator

#	Known issue or limit	Where
	hour" is read as 2 hours. The user can correct the minutes before saving.	
5	Today's recommendations need the internet; offline, the red message is shown instead and saved meals keep working.	RecipesScreen
6	A security question is weaker than an e-mail reset, and there is no lock-out after many wrong answers. Answers are hashed and not case-sensitive.	AuthService
7	Nested recipes are only built in memory (RecipeDemo and the tests); saved_recipes has no table for sub-recipes.	RecipeCalculator
8	UndoStack and CookingQueue run in RecipeDemo and the tests; the pages use Maps and Lists.	util
9	Only the free test key 1 is used, which is allowed for development and class use only.	MealDbClient
10	plate_n_platter.db is opened by a relative path, so it is created in the folder the app is started from. The old 8-table platenplatter.db is not upgraded; import the CSV files again.	DatabaseConnector
11	The streak counts days with a planned meal, not days the meal was eaten.	StreakCalculator
12	The import reads the whole file into memory (fine for tens of thousands of rows, not millions).	CsvImportTask
13	The database runs in WAL mode with synchronous = NORMAL: a power cut can lose the last save but cannot corrupt the file.	DatabaseConnector